



Intro to 333

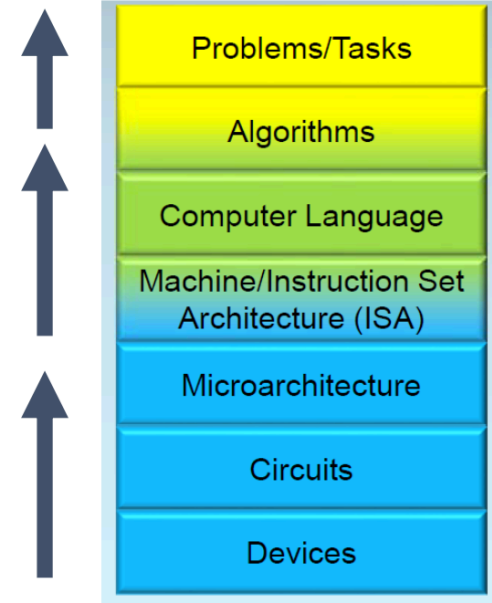
COMPUTER
ARCHITECTURE

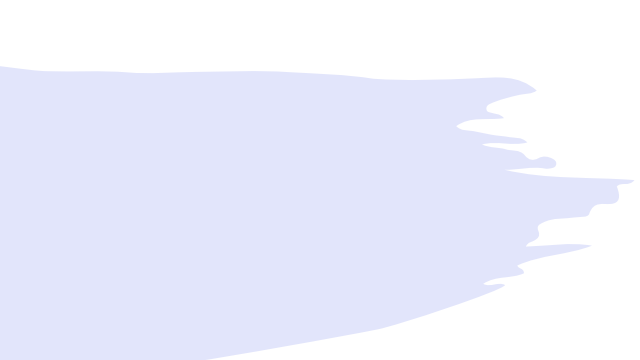
Computing Systems have multiple layers (abstractions)

CPE 202/ 203/ 357

CPE 233/ 333 / 315/ 329

CPE 133 /441



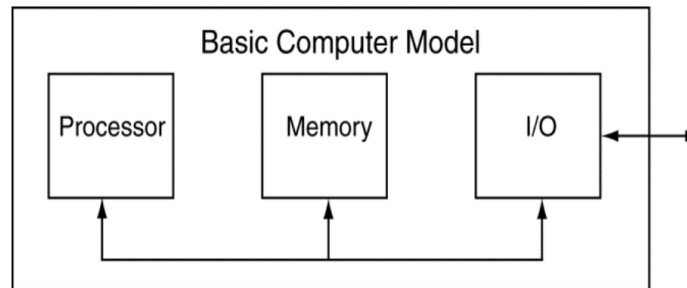


CPE 333 Intro Comments

- Course Topics
 - Computer architecture
 - Pipelining
 - Memory hierarchy (caches)
 - Virtual Memory
 - Data parallelism and thread parallelism (multicore/GPU)
 - I/O, exceptions
 - Performance Measurements
 - Energy/Power/Area Tradeoffs
- CPE 333 Materials/Topics
 - RISC-V ISA
 - OTTER uArch
- **Things to consider**
 - It's a design course, memorization won't help
 - Don't fall behind (you won't catch up)
 - Learn the material (I'll test you on what you know)

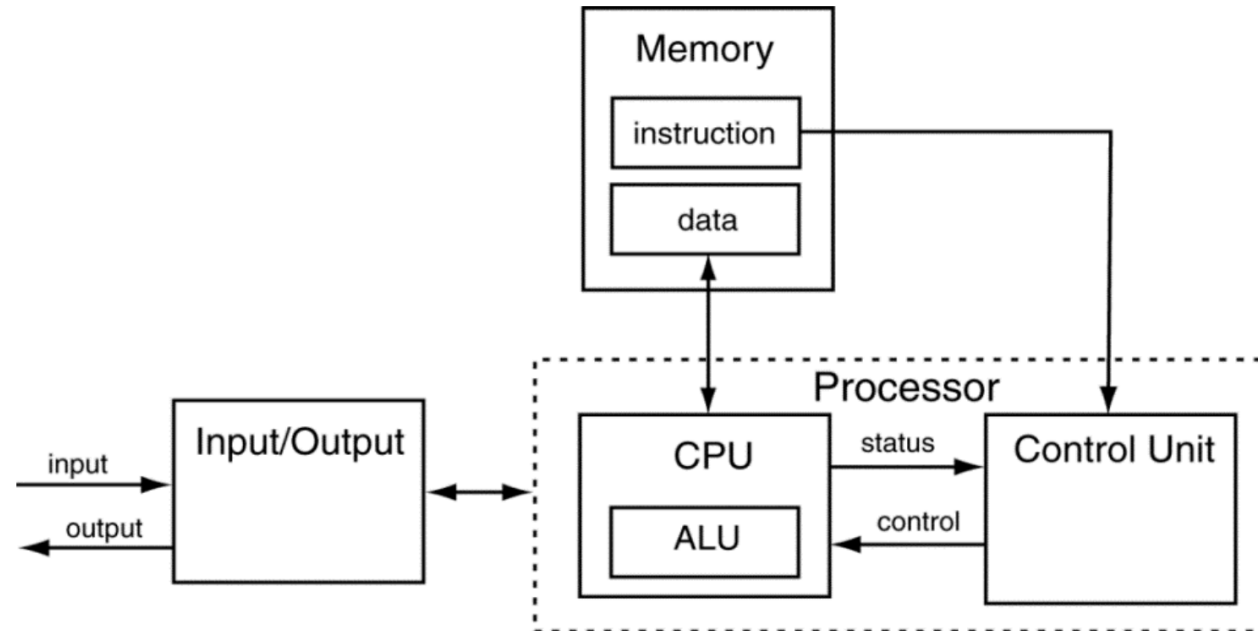
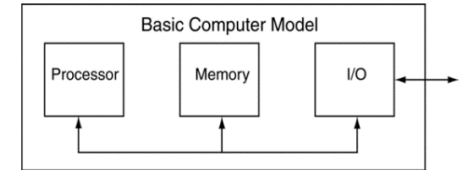
CPE 233 Review

- What is a computer?
 - A computer is a device that sequentially executes a stored program
- Von Neumann Model (accepted computer high-level architecture)



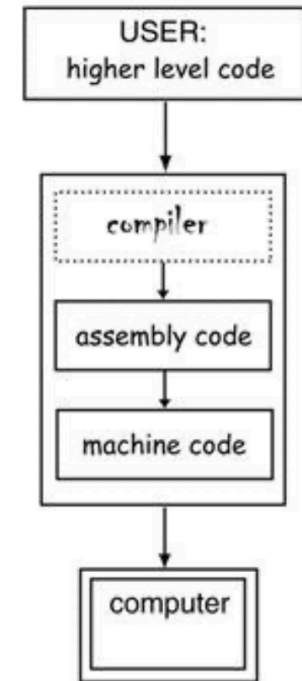
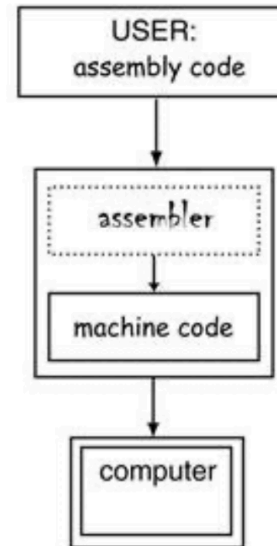
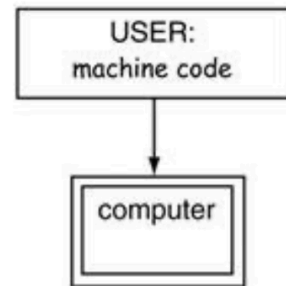
CPE 233 Review

- Lower-level computer model



CPE 233 Review

- Levels of programming

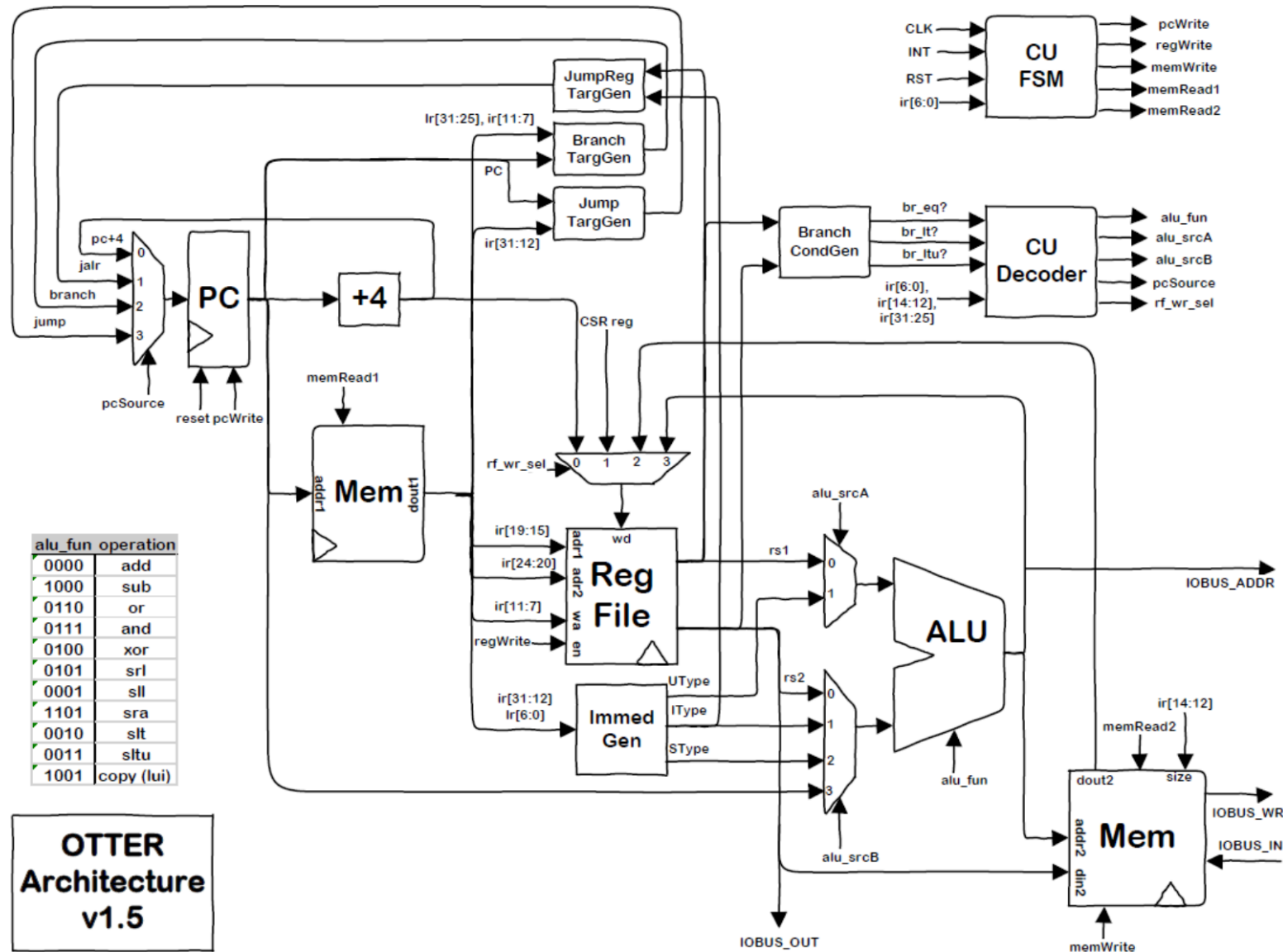




RISC-V

- Open-source, license-free, royalty-free ISA specification
- Rapid growth in industry and academia
- Both proprietary and open-source core implementations
- Supported by growing shared software ecosystem
- Appropriate for all levels of computing systems, from microcontrollers to supercomputers (32, 64, and 128 bit variants, and 16-bit compressed format)
- Standard maintained by non-profit RISC-V Foundation (<https://riscv.org/specifications/>)

OTTER: Our two Thirty Three Engineered RISC



	OTTER (RV32I)	RAT
ISA and organization	32-bit processor (2-3 cycle)	8-bit processor (2-cycle)
	37 base instructions, 26 pseudo instructions	36 instructions
	No rotate in base	Rotate Instructions
	no condition flags	condition flags
	Memory Mapped IO	Programmed IO
	Software manages stack	Hardware partially manages stack
Software tools	Spike, Venus, etc Simulators (no built-in interrupt support)	RAT simulator
	GCC tool flow Assembly and C-Code	RAT Compiler
	Standard C library	?

RISCV ISA (see cheat sheets)

OTTER (RV32I) Assembly Instructions

1 Base Instructions

Format	Base Instruction	Name	Description (in Verilog)
U	lui rd,imm	Load Upper Immediate	$R[rd] = \{imm, 12'b0\}$
U	auipc rd,imm	Add Upper Immediate to PC	$R[rd] = PC + \{imm, 12'b0\}$
J	jal rd,imm	Jump and Link	$R[rd] = PC + 4;$ $PC = PC + \{imm, 1'b0\}$
I	jalr rd,rs1,imm	Jump and Link Register	$R[rd] = PC + 4;$ $PC = R[rs1] + imm$
B	beq rs1,rs2,imm	Branch Equal	$\text{if}(R[rs1] == R[rs2]) \text{ PC} = PC + \{imm, 1'b0\}$
B	bne rs1,rs2,imm	Branch Not Equal	$\text{if}(R[rs1] != R[rs2]) \text{ PC} = PC + \{imm, 1'b0\}$
B	blt rs1,rs2,imm	Branch Less Than	$\text{if}(R[rs1] < R[rs2]) \text{ PC} = PC + \{imm, 1'b0\}$
B	bge rs1,rs2,imm	Branch Greater than Equal	$\text{if}(R[rs1] \geq R[rs2]) \text{ PC} = PC + imm, 1'b0$
B	bltu rs1,rs2,imm	Branch Less Than Unsigned	$\text{if}(R[rs1] < R[rs2]) \text{ PC} = PC + imm, 1'b0$
B	bgeu rs1,rs2,imm	Branch Greater than Equal Unsigned	$\text{if}(R[rs1] \geq R[rs2]) \text{ PC} = PC + imm, 1'b0$
I	lb rd,imm(rs1)	Load Byte	$R[rd] = 24'bM[R[rs1] + imm](7), M[R[rs1] + imm](7:0)$
I	lh rd,imm(rs1)	Load Half	$R[rd] = 16'bM[R[rs1] + imm](7), M[R[rs1] + imm](15:0)$
I	lw rd,imm(rs1)	Load Word	$R[rd] = M[R[rs1] + imm]$
I	lbu rd,imm(rs1)	Load Byte Unsigned	$R[rd] = 24'b0, M[R[rs1] + imm](7:0)$
I	lhu rd,imm(rs1)	Load Half Unsigned	$R[rd] = 16'b0, M[R[rs1] + imm](15:0)$
S	sb rs2,imm(rs1)	Store Byte	$M[R[rs1] + imm](7:0) = R[rs2](7:0)$
S	sh rs2,imm(rs1)	Store Half	$M[R[rs1] + imm](15:0) = R[rs2](15:0)$
S	sw rs2,imm(rs1)	Store Word	$M[R[rs1] + imm] = R[rs2]$
I	addi rd,rs1,imm	Add Immediate	$R[rd] = R[rs1] + imm$
I	slti rd,rs1,imm	Set Less Than Immediate	$R[rd] = (\$signed(R[rs1]) < \$signed(imm)) ? 1 : 0$
I	sltiu rd,rs1,imm	Set Less Than Immediate Unsigned	$R[rd] = (R[rs1] < imm) ? 1 : 0$
I	xori rd,rs1,imm	Xor Immediate	$R[rd] = R[rs1] \wedge imm$
I	ori rd,rs1,imm	Or Immediate	$R[rd] = R[rs1] \vee imm$
I	andi rd,rs1,imm	And Immediate	$R[rd] = R[rs1] \& imm$
I	slli rd,rs1,imm	Shift Left Logical Immediate	$R[rd] = R[rs1] << imm(4:0)$
I	srl1 rd,rs1,imm	Shift Right Logical Immediate	$R[rd] = R[rs1] >> imm(4:0)$
I	srai rd,rs1,imm	Shift Right Arithmetic Immediate	$R[rd] = \$signed(R[rs1]) >>> imm(4:0)$
R	add rd,rs1,rs2	Add	$R[rd] = R[rs1] + R[rs2]$
R	sub rd,rs1,rs2	Subtract	$R[rd] = R[rs1] - R[rs2]$
R	sll rd,rs1,rs2	Shift Left Logical	$R[rd] = R[rs1] << R[rs2](4:0)$
R	slt rd,rs1,rs2	Set Less Than	$R[rd] = (\$signed(R[rs1]) < \$signed(R[rs2])) ? 1 : 0$
R	sltu rd,rs1,rs2	Set Less Than Unsigned	$R[rd] = (R[rs1] < R[rs2]) ? 1 : 0$
R	xor rd,rs1,rs2	Xor	$R[rd] = R[rs1] \wedge R[rs2]$
R	srl rd,rs1,rs2	Shift Right Logical	$R[rd] = R[rs1] >> R[rs2](4:0)$
R	sra rd,rs1,rs2	Shift Right Arithmetic	$R[rd] = \$signed(R[rs1]) >>> R[rs2](4:0)$
R	or rd,rs1,rs2	Or	$R[rd] = R[rs1] \vee R[rs2]$
R	and rd,rs1,rs2	And	$R[rd] = R[rs1] \& R[rs2]$
csrww rd,csr,rs1			$R[rd] = csr, csr = R[rs1]$
mret			$PC = mepc$

Table 1: OTTER (RV32I) Instructions.

2 Psuedo Instructions

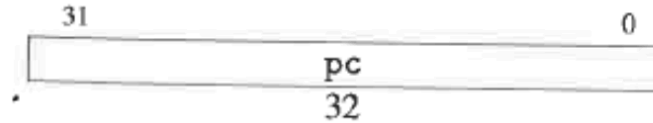
pseudoinstruction	Base Instruction(s)	Meaning
la rd, symbol	auipc rd, delta[31:12] + delta[11] addi rd, rd, delta[11:0]	Load absolute address, where $\text{delta} = \text{symbol} - \text{pc}$
lw rd, symbol	auipc rd, delta[31:12] + delta[11] lw rd, delta[11:0](rd)	Load global
sw rd, symbol, rt	auipc rt, delta[31:12] + delta[11] sw rd, delta[11:0](rt)	Store global
<i>The base instructions use pc-relative addressing, so the linker subtracts pc from symbol to get delta. The linker adds delta[11] to the 30-bit high part, countenacting sign extension of the 12-bit low part.</i>		
nop	addi x0, x0, 0	No operation
li rd, immediate	<i>Myriad sequences</i>	Load immediate
mv rd, rs	addi rd, rs, 0	Copy register
not rd, rs	xori rd, rs, -1	One's complement
neg rd, rs	sub rd, x0, rs	Two's complement
seqz rd, rs	sltiu rd, rs, 1	Set if = zero
snez rd, rs	sltu rd, x0, rs	Set if $\neq$ zero
sltz rd, rs	slt rd, rs, x0	Set if < zero
sgtz rd, rs	slt rd, x0, rs	Set if > zero
beqz rs, offset	beq rs, x0, offset	Branch if = zero
bnez rs, offset	bne rs, x0, offset	Branch if $\neq$ zero
blez rs, offset	bge x0, rs, offset	Branch if $\leq$ zero
bgez rs, offset	bge rs, x0, offset	Branch if $\geq$ zero
bltz rs, offset	blt rs, x0, offset	Branch if < zero
bgtz rs, offset	blt x0, rs, offset	Branch if > zero
bgt rs, rt, offset	blt rt, rs, offset	Branch if >
ble rs, rt, offset	bge rt, rs, offset	Branch if $\leq$
bgtru rs, rt, offset	bltu rt, rs, offset	Branch if >, unsigned
bleu rs, rt, offset	bgeu rt, rs, offset	Branch if $\leq$, unsigned
j offset	jal x0, offset	Jump
jal offset	jal x1, offset	Jump and link
jr rs	jalr x0, 0(rs)	Jump register
jalr rs	jalr x1, 0(rs)	Jump and link register
ret	jalr x0, 0(x1)	Return from subroutine
call offset	auipc x1, offset[31:12] + offset[11] jalr x1, offset[11:0](x1)	Call far-away subroutine
tail offset	auipc x6, offset[31:12] + offset[11] jalr x0, offset[11:0](x6)	Tail call far-away subroutine
fence	fence iorw, iorw	Fence on all memory and I/O

Table 2: RISC-V pseudoinstructions.

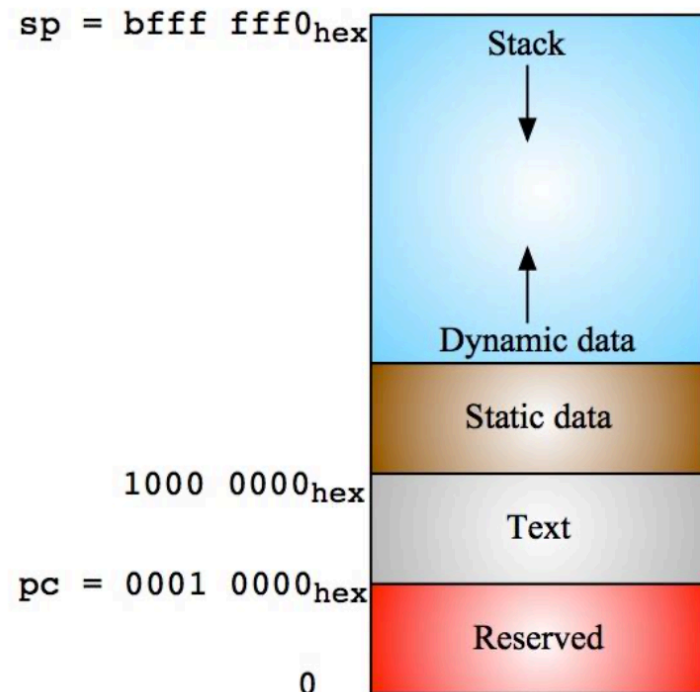
OTTER (RV32I)

Register File:

Program Counter:



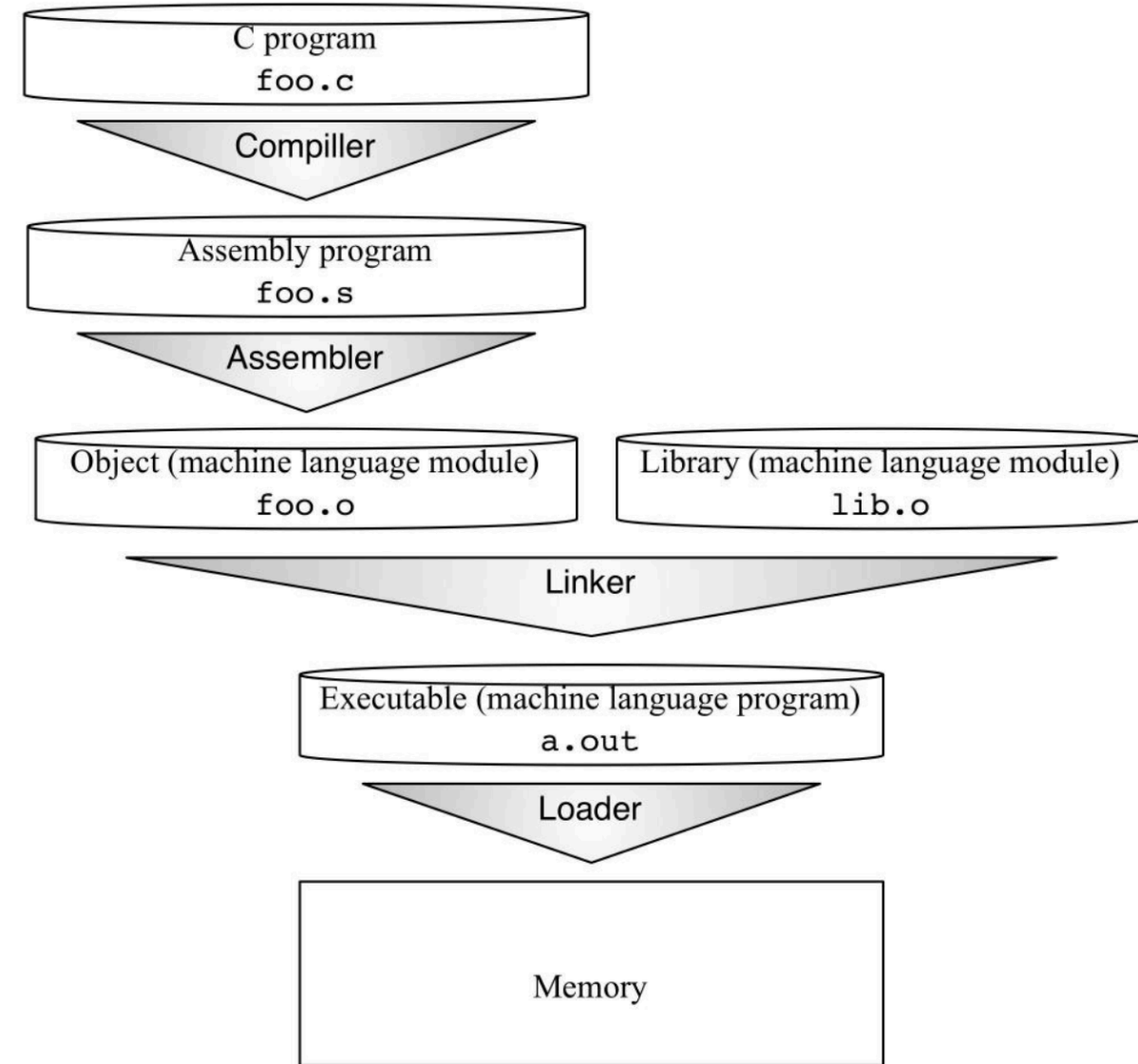
Memory:



31		0
	x0 / zero	
	x1 / ra	
	x2 / sp	
	x3 / gp	
	x4 / tp	
	x5 / t0	
	x6 / t1	
	x7 / t2	
	x8 / s0 / fp	
	x9 / s1	
	x10 / a0	
	x11 / a1	
	x12 / a2	
	x13 / a3	
	x14 / a4	
	x15 / a5	
	x16 / a6	
	x17 / a7	
	x18 / s2	
	x19 / s3	
	x20 / s4	
	x21 / s5	
	x22 / s6	
	x23 / s7	
	x24 / s8	
	x25 / s9	
	x26 / s10	
	x27 / s11	
	x28 / t3	
	x29 / t4	
	x30 / t5	
	x31 / t6	
	32	

Software Details

- Assembly Constructs Cheat sheets
- RISC-V support is upstream in GNU toolchain (GCC, GDB)
- Most students used Virtualbox Ubuntu image with precompiled cross compiler (also very easy on MAC with homebrew)
- test_all.s validation program

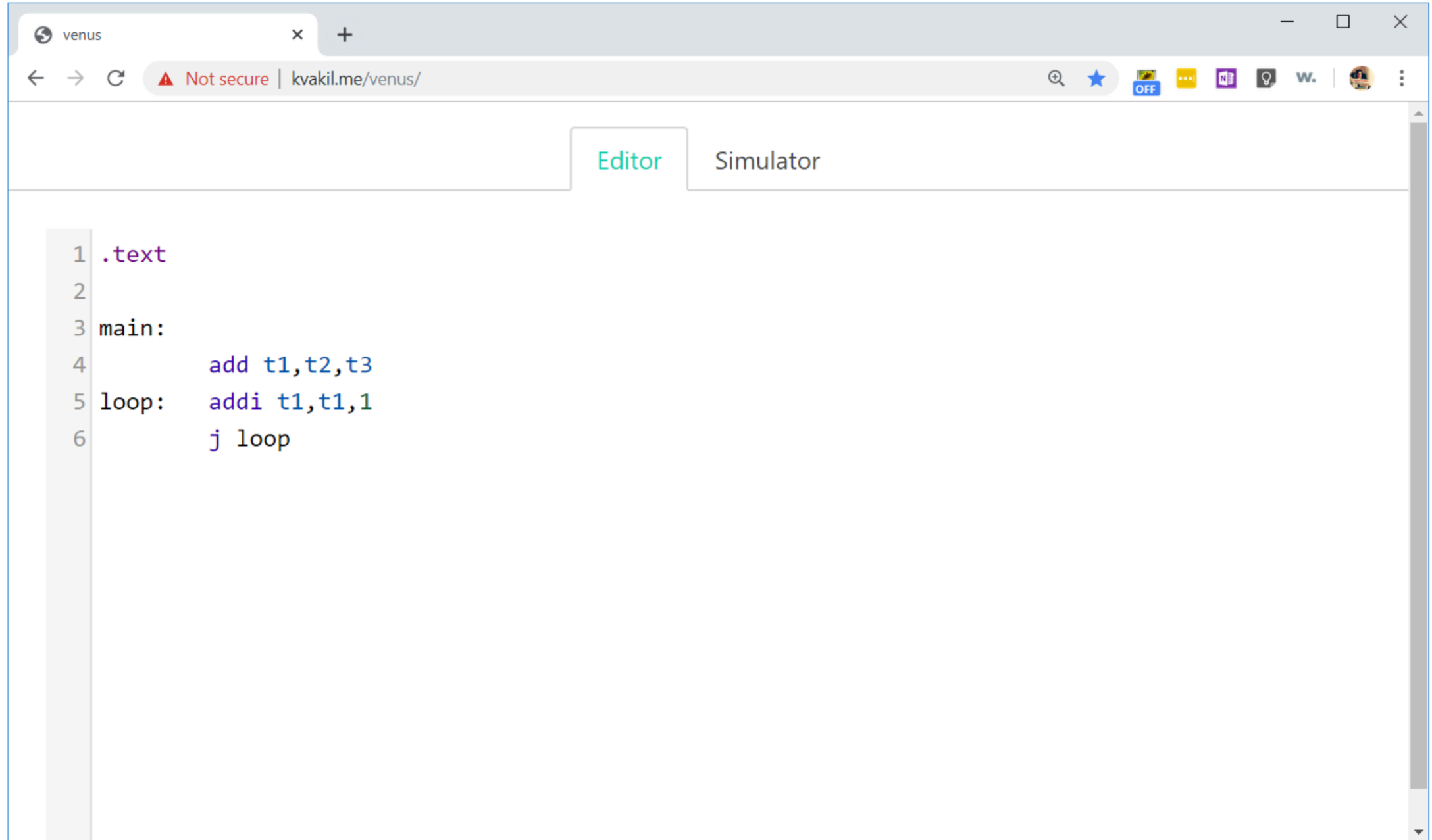




Software tools

- Simulators: Venus/RARS/Spike
- GCC/GNU Toolchain
- Virtual box – Ubuntu image with prebuilt cross compiler

Assembly Programming -Venus



The screenshot shows a web browser window with a single tab titled 'venus'. The address bar shows the URL 'kvakil.me/venus/' with a 'Not secure' warning. The browser's toolbar includes navigation buttons, a search icon, a star for bookmarks, and various extension icons. The main content area has two tabs: 'Editor' (active) and 'Simulator'. The 'Editor' tab contains a text editor with the following assembly code:

```
1 .text
2
3 main:
4     add t1,t2,t3
5 loop: addi t1,t1,1
6     j loop
```



Run speed 0.5 inst/sec

Edit Execute

riscv1.asm

```
1 .eqv
2
3
4 .text
5 main:
6     add t1,t2,t3
7 loop: addi t1,t1,1
8     j loop
9
10
11
```

Line: 11 Column: 1 ☒ Show Line Numbers

Messages

Run I/O

Clear

Assemble: assembling C:\Users\jcallene\Downloads\riscv1.asm

Registers

Floating Point

Control and Status

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7fffffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x0000013a
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400004

